Mixture models

Frank Edwards

Overdispersion

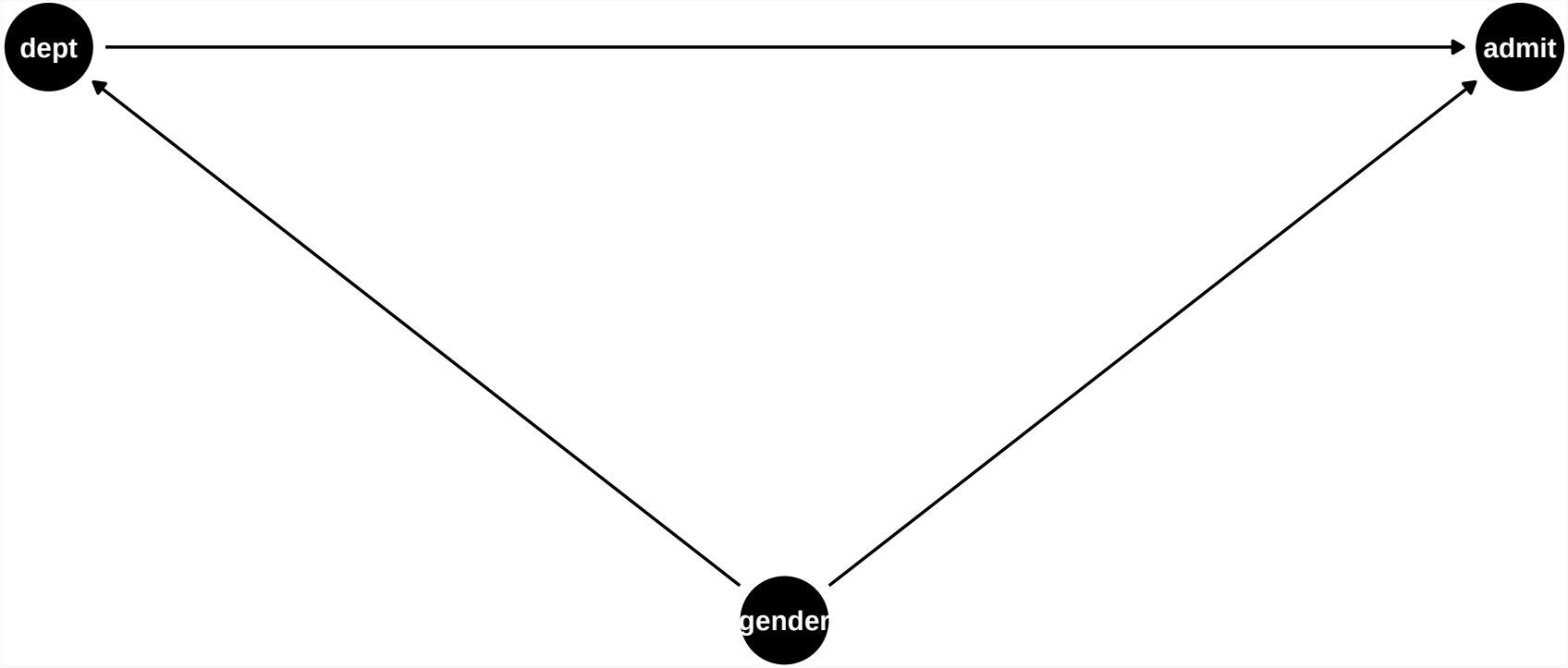
Admissions

```
data("UCBadmit")
d<-UCBadmit
d
```

```
   dept applicant.gender admit reject applications
1     A             male   512    313          825
2     A           female    89     19          108
3     B             male   353    207          560
4     B           female    17      8           25
5     C             male   120    205          325
6     C           female   202    391          593
7     D             male   138    279          417
8     D           female   131    244          375
9     E             male    53    138          191
10    E           female    94    299          393
11    F             male    22    351          373
12    F           female    24    317          341
```

Recall our DAG

```
dag0 <- dagify(admit~gender + dept,
               dept ~ gender)
ggdag(dag0) + theme_dag()
```



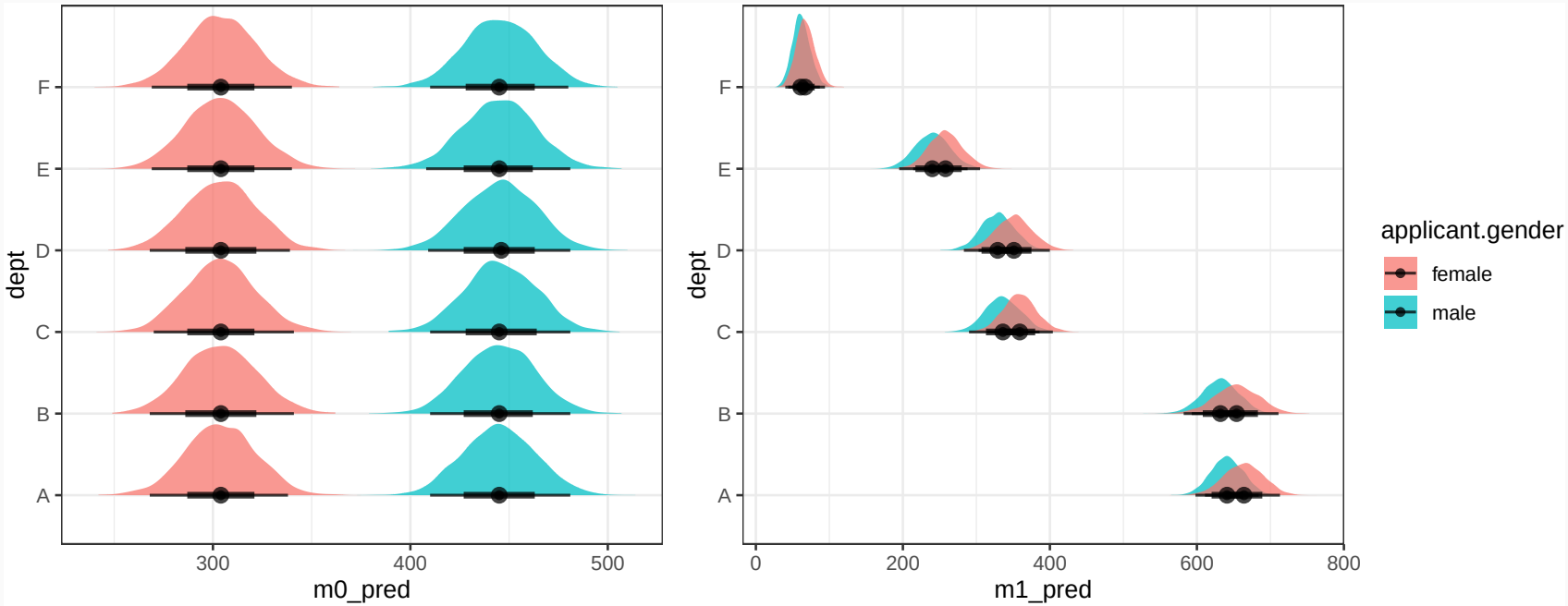
Let’s compare two approaches

```
# model without department
m0 <- brm(admit | trials(applications) ~ applicant.gender,
          data = d,
          family = "binomial")
# model with department
m1 <- brm(admit | trials(applications) ~ applicant.gender + dept,
          data = d,
          family = "binomial")
# df for prediction and visualization
sim_dat <- expand_grid(dept = unique(d$dept),
                       applicant.gender = unique(d$applicant.gender),
                       applications = 1000)
sim_dat_m0 <- sim_dat |>
  add_predicted_draws(m0, value = "m0_pred")
sim_dat_m1 <- sim_dat |>
  add_predicted_draws(m1, value = "m1_pred")
```

Expected admissions for 1000 applicants

```
p1 <- ggplot(sim_dat_m0, aes(x = m0_pred, fill = applicant.gender,
                             y = dept)) +
  stat_slabinterval(alpha = 0.75)
p2 <- ggplot(sim_dat_m1, aes(x = m1_pred, fill = applicant.gender,
                             y = dept)) +
  stat_slabinterval(alpha = 0.75)

p1+p2 + plot_layout(guides="collect")
```



What does the parameter for ‘department’ do in the model?

```
posterior_interval(m1)
```

```
                             2.5%       97.5%
b_Intercept             0.4931537   0.8713875
b_applicant.gendermale -0.2539323   0.0535443
b_deptB                -0.2674630   0.1769511
b_deptC                -1.4688433  -1.0530967
b_deptD                -1.5029374  -1.0929457
b_deptE                -1.9781192  -1.4924211
b_deptF                -3.6580641  -2.9880272
Intercept              -0.7233857  -0.5695207
lprior                 -1.9722322  -1.9514815
lp__                  -54.6912428 -47.3836987
```

What if each department had their own probability of admission?

Our binomial regression works like this

$$admit_i \sim \text{Binomial}(applications_i, p_i)$$

$$\text{logit}(p_i) = \beta_0 + \beta_1 gender_i + \beta_2 dept_i$$

What if each department had their own probability of admission?

Our binomial regression works like this

$$admit_i \sim \text{Binomial}(applications_i, p_i)$$
$$\text{logit}(p_i) = \beta_0 + \beta_1 gender_i + \beta_2 dept_i$$

the logit of admission probability p is a linear function of gender and department.

What if each department had their own probability of admission?

Our binomial regression works like this

$$admit_i \sim \text{Binomial}(applications_i, p_i)$$
$$\text{logit}(p_i) = \beta_0 + \beta_1 gender_i + \beta_2 dept_i$$

the logit of admission probability p is a linear function of gender and department.

We know we underestimate the variability of the data when we exclude information on department.

Overdispersion

Overdispersion occurs when there is more variability in the outcome than what is expected by the model.

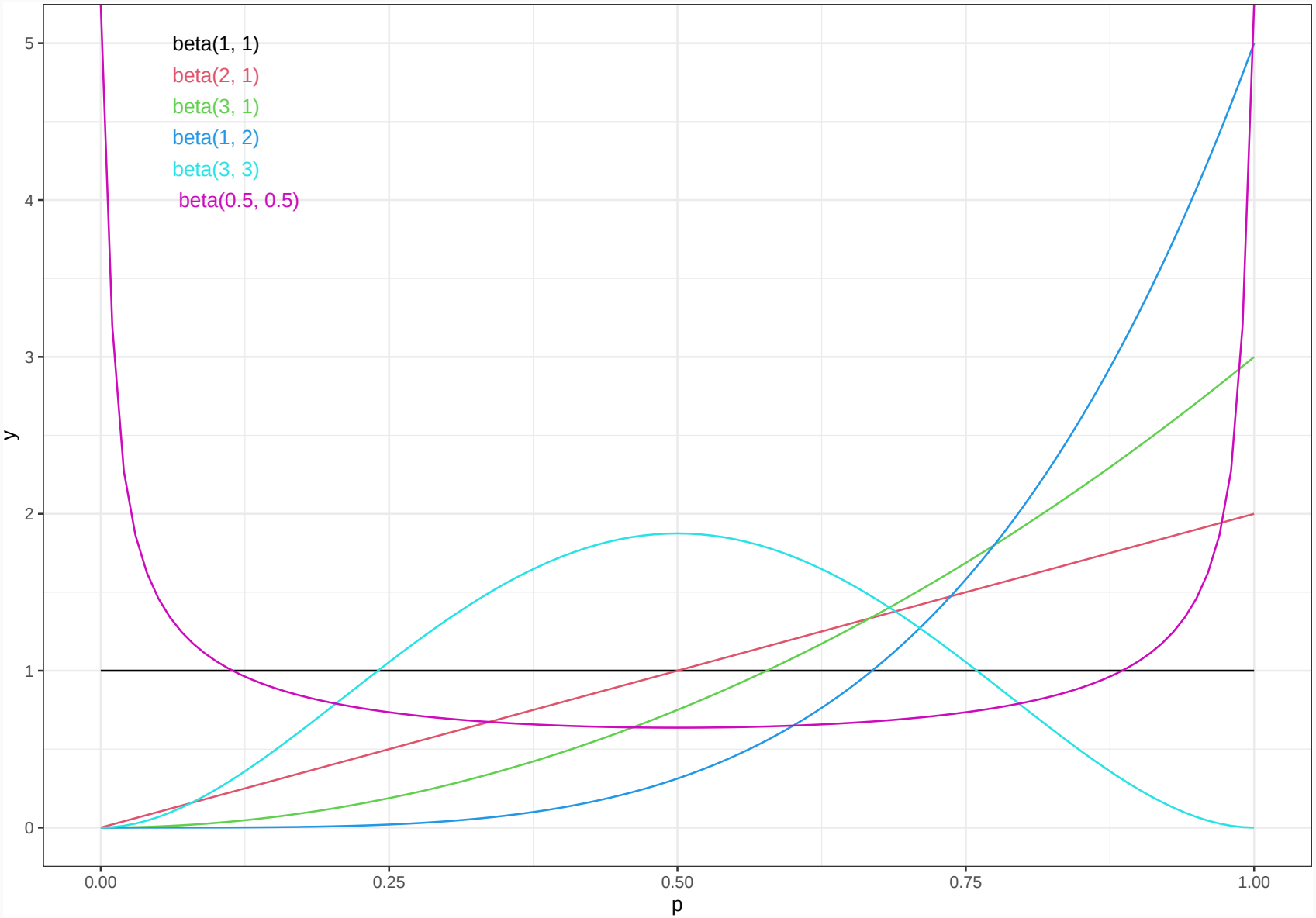
An alternative approach

Perhaps we don't have information on department, but suspect that admission probabiliy p is likely to vary across units.

Let's create an explicit probability model for p to allow it to vary across units

$$p \sim \text{Beta}(a, b)$$

The Beta distribution



Let's assume a different model

A Beta Binomial model allows for heterogeneity in p, the probability of admission, across rows of the data.

$$p \sim \text{Beta}(\mu, \phi)$$

$$y \sim \text{Binomial}(n, p)$$

$$\text{logit}(\mu) = \beta_0 + \beta_1 gender$$

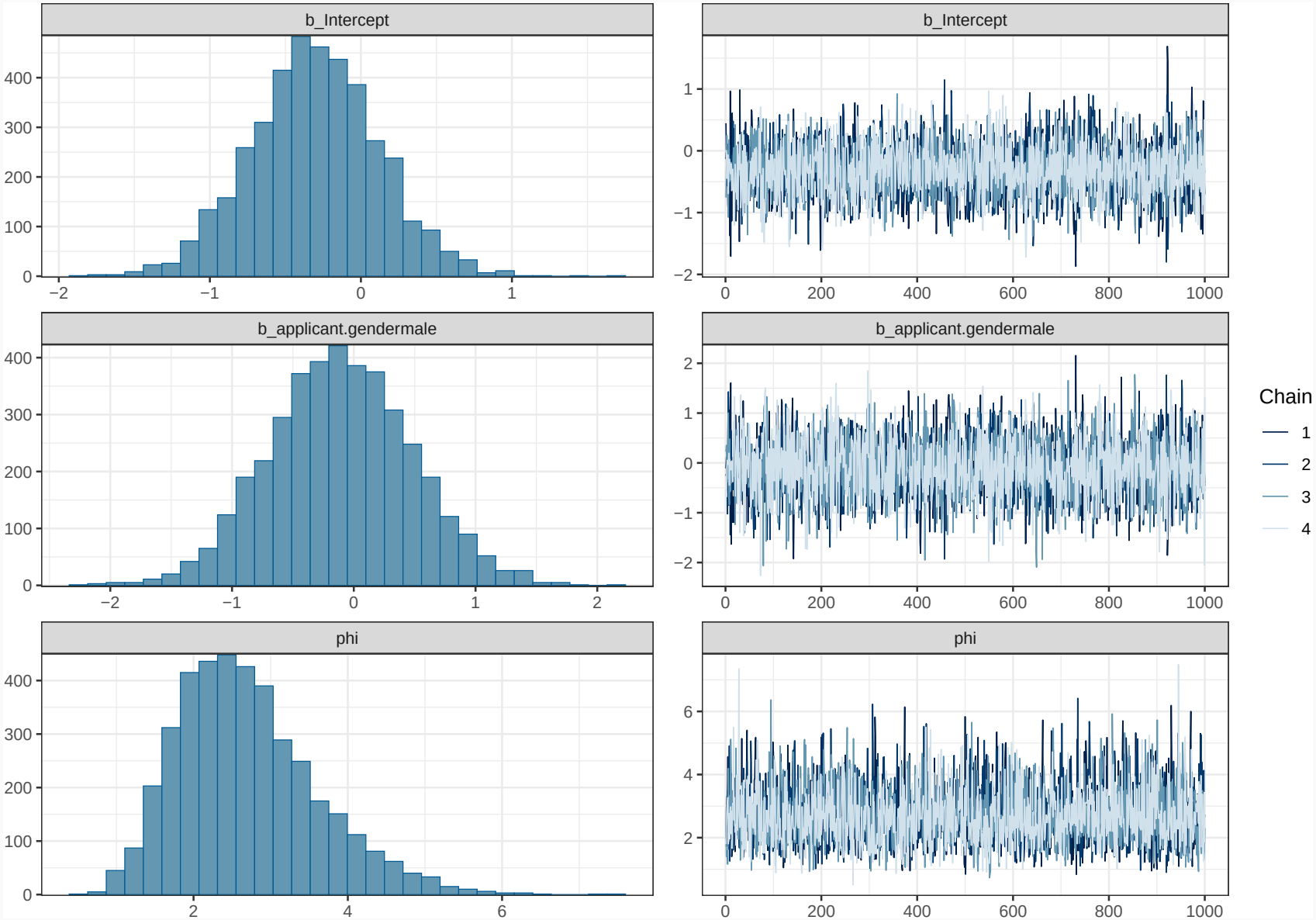
Where ϕ is a parameter that adjusts for over-dispersion

Estimation

```
# priors for intercept, beta, and phi
# phi is strictly non-negative
priors <- c(set_prior("normal(0,1)", class = "Intercept"),
            set_prior("normal(0,2)", class = "b"),
            set_prior("exponential(1)", class = "phi"))

m_admit <- brm(admit | trials(applications) ~
                 applicant.gender,
               family = "beta_binomial",
               data = d,
               prior = priors,
               cores = 4)
```

Convergence



Output

```
m_admit
```

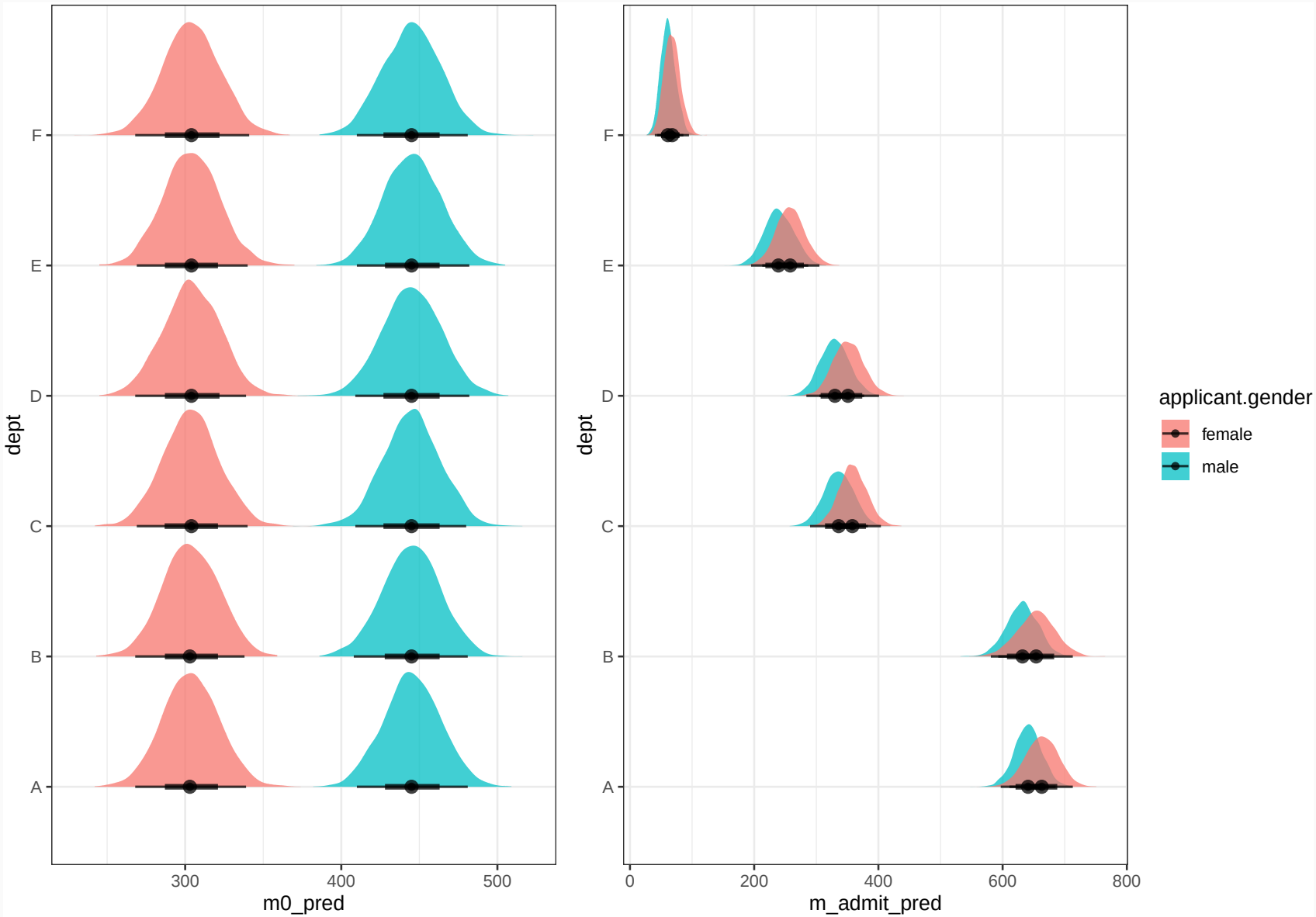
```
 Family: beta_binomial
  Links: mu = logit
Formula: admit | trials(applications) ~ applicant.gender
   Data: d (Number of observations: 12)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
                    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
Intercept              -0.31      0.42    -1.13     0.54 1.00     3053
applicant.gendermale   -0.11      0.59    -1.24     1.05 1.00     3608
                    Tail_ESS
Intercept               2585
applicant.gendermale    2220

Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi     2.69      0.91     1.26     4.79 1.00     2990     2830

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

What the Beta mixture achieves. Remember: no department control!



Overdispersion in count models

The Poisson model

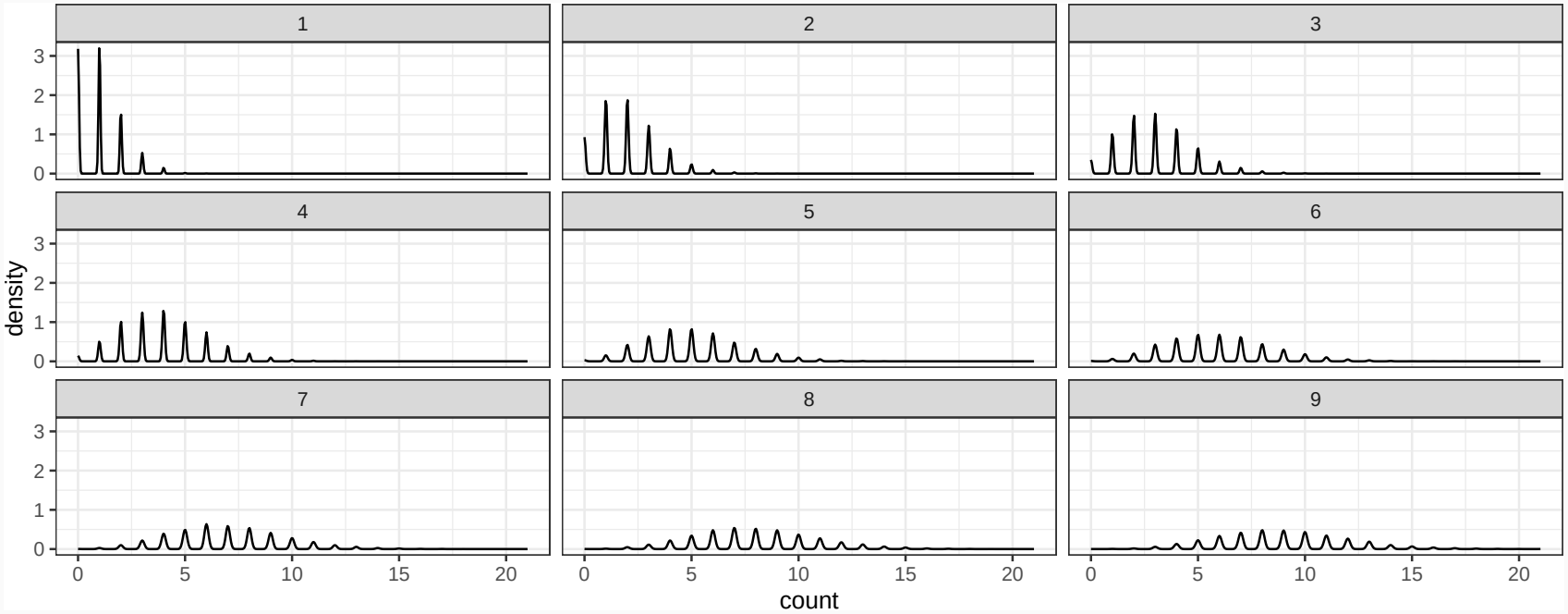
$$y \sim Poisson(\lambda)$$

$$E(y) = \lambda$$

$$Var(y) = \lambda$$

The Poisson distribution

```
ggplot(pois_demo, aes(x=count)) +
  geom_density(adjust = 1/4) +
  facet_wrap(~lambda)
```



Overdispersion

A variable is said to be *overdispersed* if the variance exceeds what is expected by the model.

A count is overdispersed if the variance exceeds the mean (Poisson) or the variance exceeds $np(1-p)$ (Binomial).

What overdispersion looks like in practice

```
### make total deaths and pop per county
fe<-read_csv("./fe_demo.csv") |>
  group_by(fips, state) |>
  summarise(pop = sum(pop),
            deaths = sum(deaths)) |>
  mutate(mort_rt = deaths / pop * 1e5) # empirical crude mortality

mean(fe$mort_rt)
```

```
[1] 5.079719
```

```
var(fe$mort_rt)
```

```
[1] 131.2199
```

Estimating an intercept-only offset Poisson model

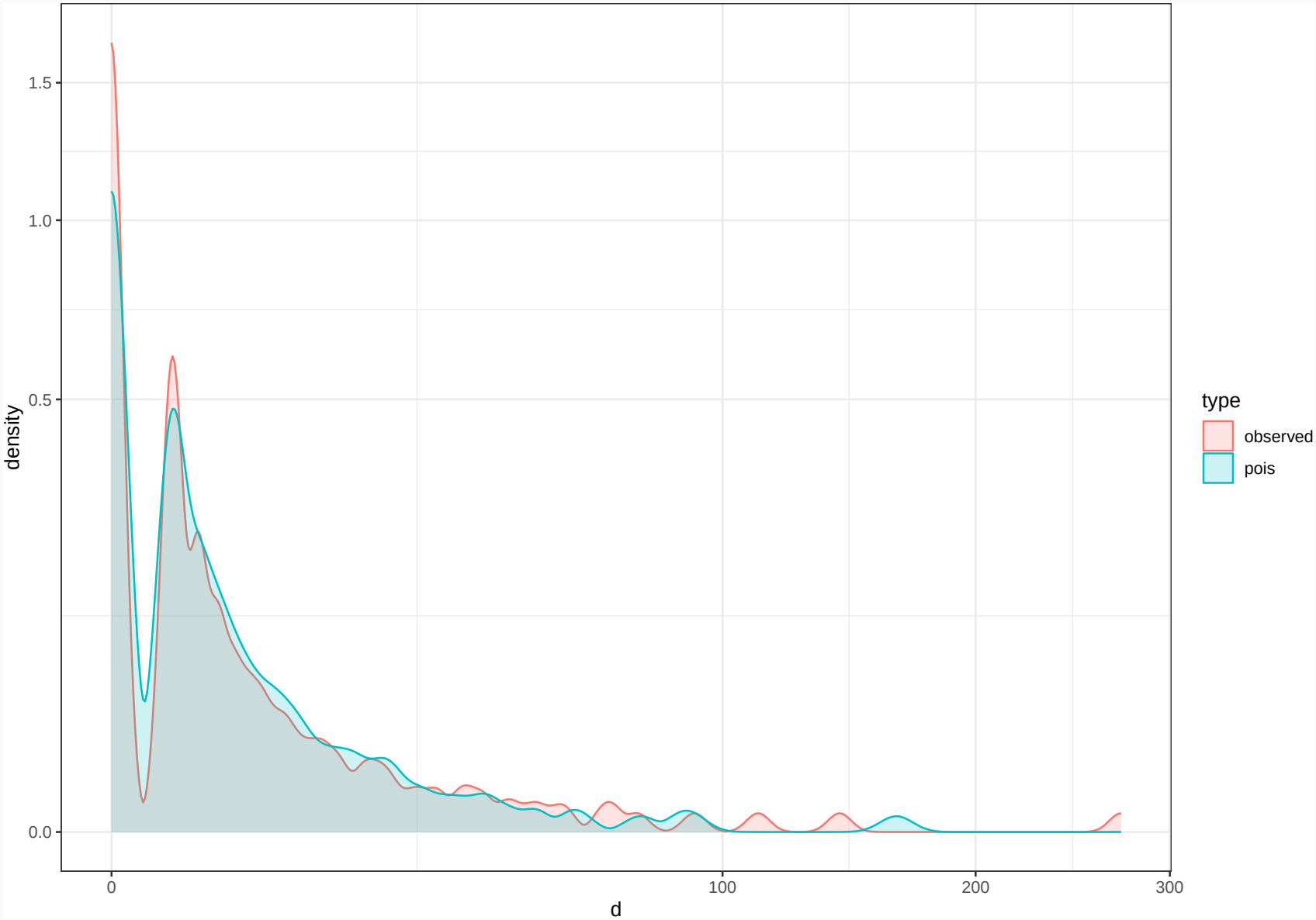
$$d_i \sim Poisson(\lambda)$$

$$\log \lambda_i = \log p_i + \alpha$$

$$\alpha \sim Normal(0, 2)$$

```
fe_0 <- brm(deaths ~ 1 + offset(log(pop)),
            prior = set_prior("normal(0,2)", class = "Intercept"),
            data = fe,
            family = "poisson",
            cores = 4)
```

Let’s compare model predictions to the observed data



Overdispersion in practice

- What if each county had a different expected mortality rate?

Overdispersion in practice

- What if each county had a different expected mortality rate?
- Overdispersion results from mixed processes, when outcomes have more than one cause or are a sum or product of processes

Overdispersion in practice

- What if each county had a different expected mortality rate?
- Overdispersion results from mixed processes, when outcomes have more than one cause or are a sum or product of processes
- Most count variables are overdispersed

Overdispersion in practice

- What if each county had a different expected mortality rate?
- Overdispersion results from mixed processes, when outcomes have more than one cause or are a sum or product of processes
- Most count variables are overdispersed
- Failing to adjust for overdispersion leads to over-confidence in posterior inference (intervals are too narrow)

Overdispersion in practice

- What if each county had a different expected mortality rate?
- Overdispersion results from mixed processes, when outcomes have more than one cause or are a sum or product of processes
- Most count variables are overdispersed
- Failing to adjust for overdispersion leads to over-confidence in posterior inference (intervals are too narrow)
- Generally safe to assume overdispersion

Addressing overdispersion

- Mixture models gamma + Poisson (negative binomial) and beta + binomial (beta binomial) models handle this problem well

Addressing overdispersion

- Mixture models gamma + Poisson (negative binomial) and beta + binomial (beta binomial) models handle this problem well
- Multilevel models with observation-level intercepts also flexibly model overdispersion

How it works: a mixture model

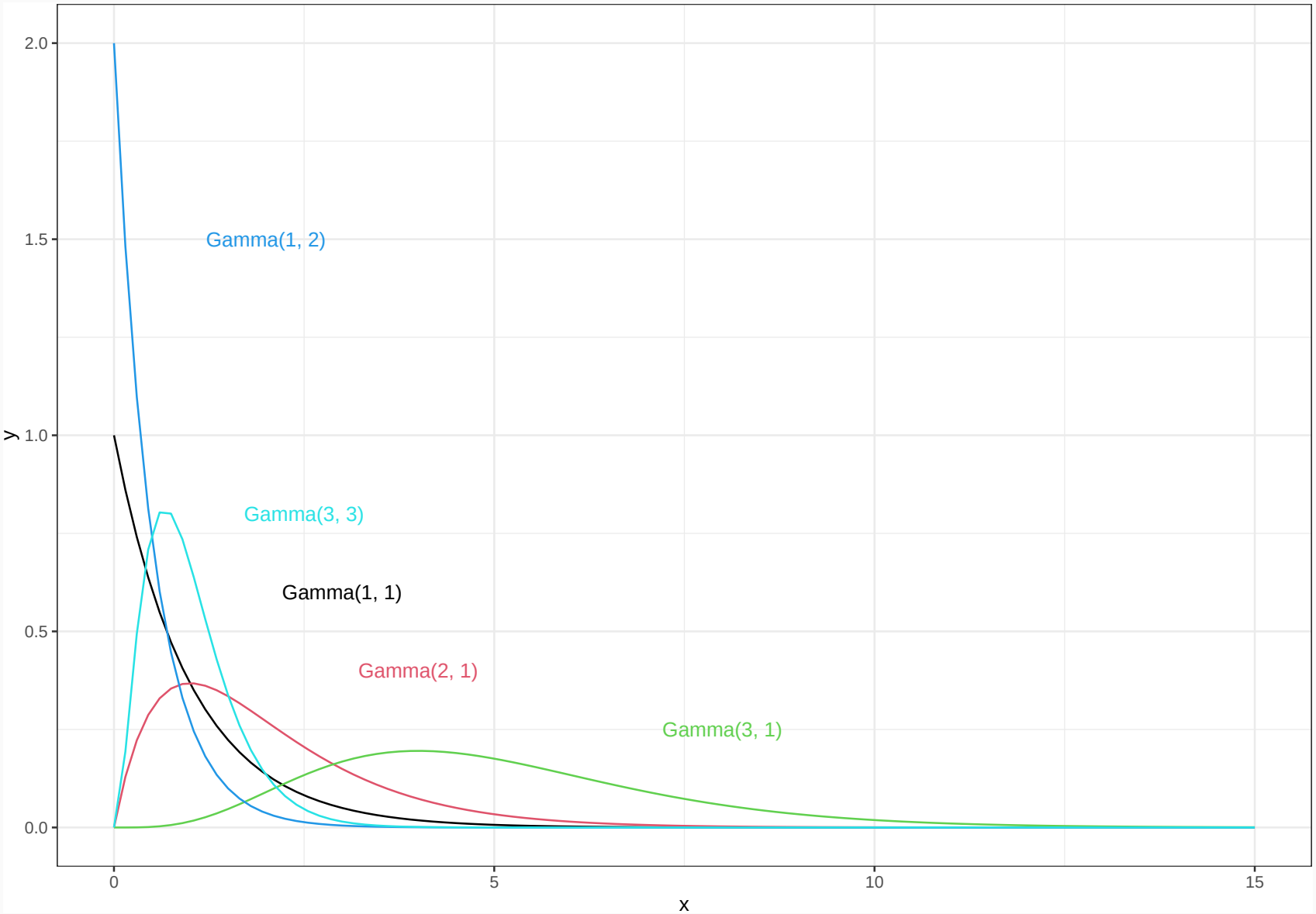
Negative Binomial models treat the Poisson parameter λ as a random variable

$$y_i \sim \text{Poisson}(\lambda_i)$$

$$\lambda_i \sim \text{Gamma}(\alpha, \theta)$$

It allows *each* observation to have it's own rate λ_i

The Gamma distribution



A mixture model

We could imagine estimating a model in two stages:

1. Estimate a Gamma regression for the event rate (λ_i) for each unit

A mixture model

We could imagine estimating a model in two stages:

1. Estimate a Gamma regression for the event rate (λ_i) for each unit
2. Estimate a Poisson regression for the event count y_i, using the λ_i parameter from step 1

Modeling overdispersion: the negative binomial model

- To allow for unmeasured variables to influence counts, we can let each Poisson variable have it's own event rate

Modeling overdispersion: the negative binomial model

- To allow for unmeasured variables to influence counts, we can let each Poisson variable have it's own event rate
- We mix the Poisson and the gamma distributions to model the the rate (Gamma) and count (Poisson) of each observation

Modeling overdispersion: the negative binomial model

- To allow for unmeasured variables to influence counts, we can let each Poisson variable have it's own event rate
- We mix the Poisson and the gamma distributions to model the the rate (Gamma) and count (Poisson) of each observation

Defining the Negative Binomial model

- We add one extra parameter to the Poisson model: a dispersion parameter ϕ that allows for extra variance in counts, effectively allowing the event rate to vary across units

$$y \sim \text{Negative Binomial}(\lambda, \phi)$$

Defining the Negative Binomial model

- We add one extra parameter to the Poisson model: a dispersion parameter ϕ that allows for extra variance in counts, effectively allowing the event rate to vary across units

$$y \sim \text{Negative Binomial}(\lambda, \phi)$$

Negative binomials can be parameterized in many different ways.

In `brms`, as $\phi \to \infty$ Negative binomial converges to Poisson (larger ϕ = less variance, smaller ϕ = more variance)

Re-estimating the police deaths model

$$d_i \sim \text{Negative Binomial}(\lambda, \phi)$$

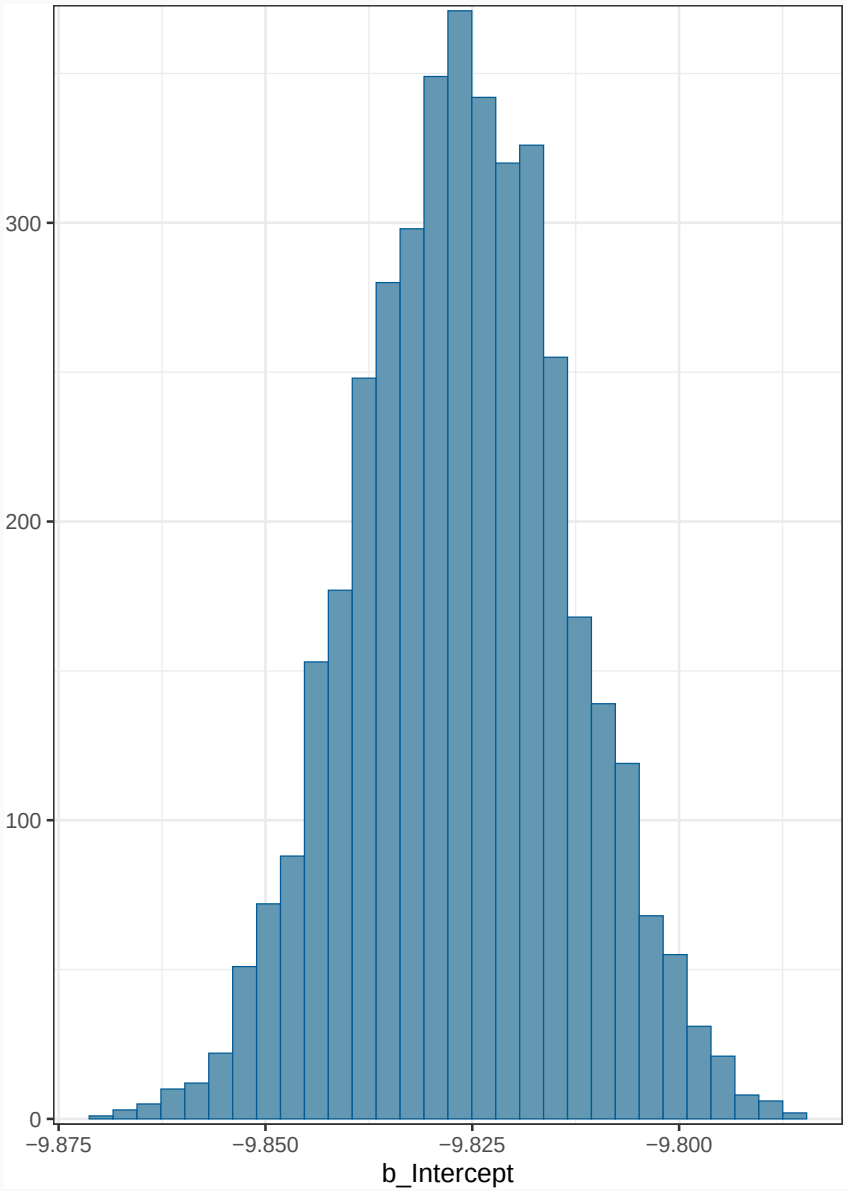
$$\log \lambda_i = \log p_i + \alpha$$

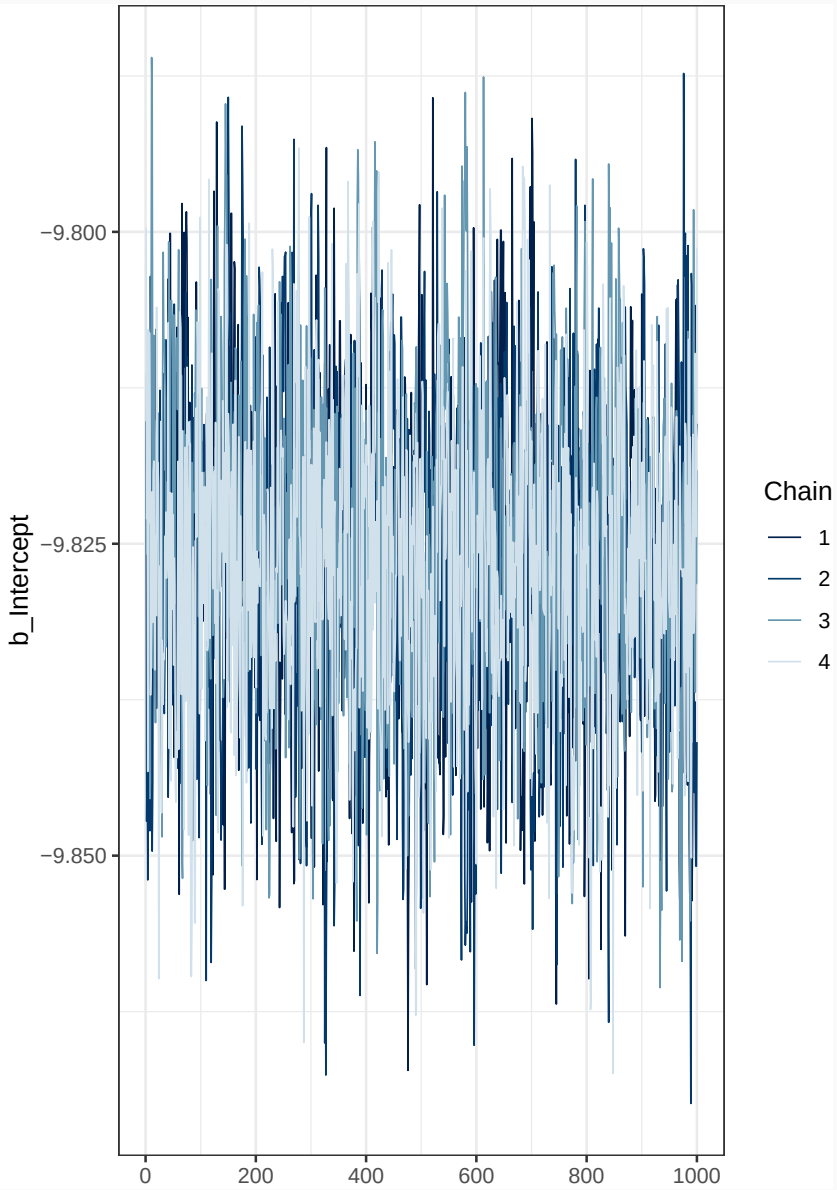
$$\alpha \sim Normal(0, 2)$$

$$\phi \sim Exponential(1)$$

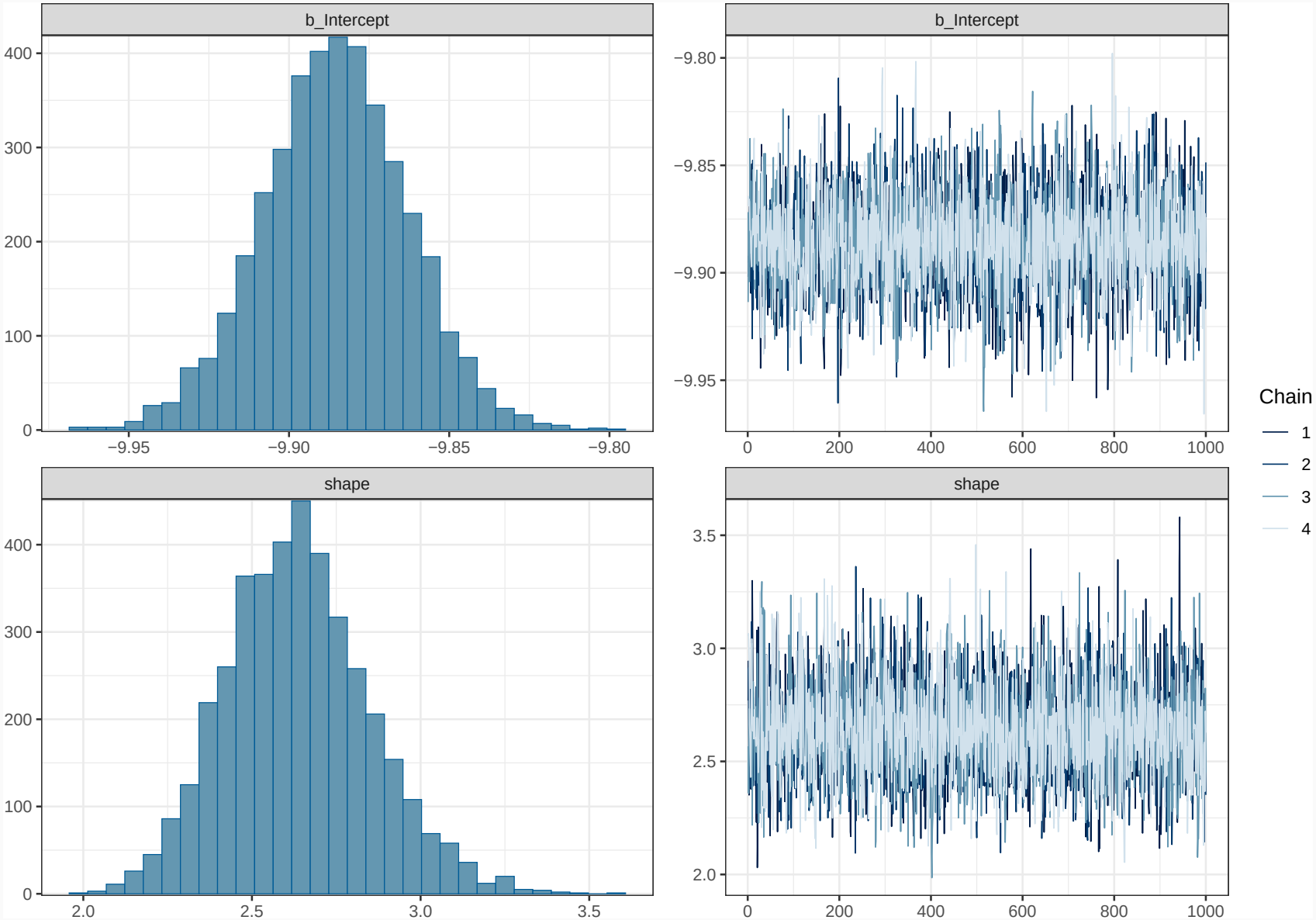
```
fe_NB0 <- brm(deaths ~ offset(log(pop)),
              family = "negbinomial",
              prior = c(set_prior("normal(0,10)", class = "Intercept"),
                        set_prior("exponential(1)", class = "shape")),
              data = fe,
              cores = 4)
```

Examining the posteriors: Poisson

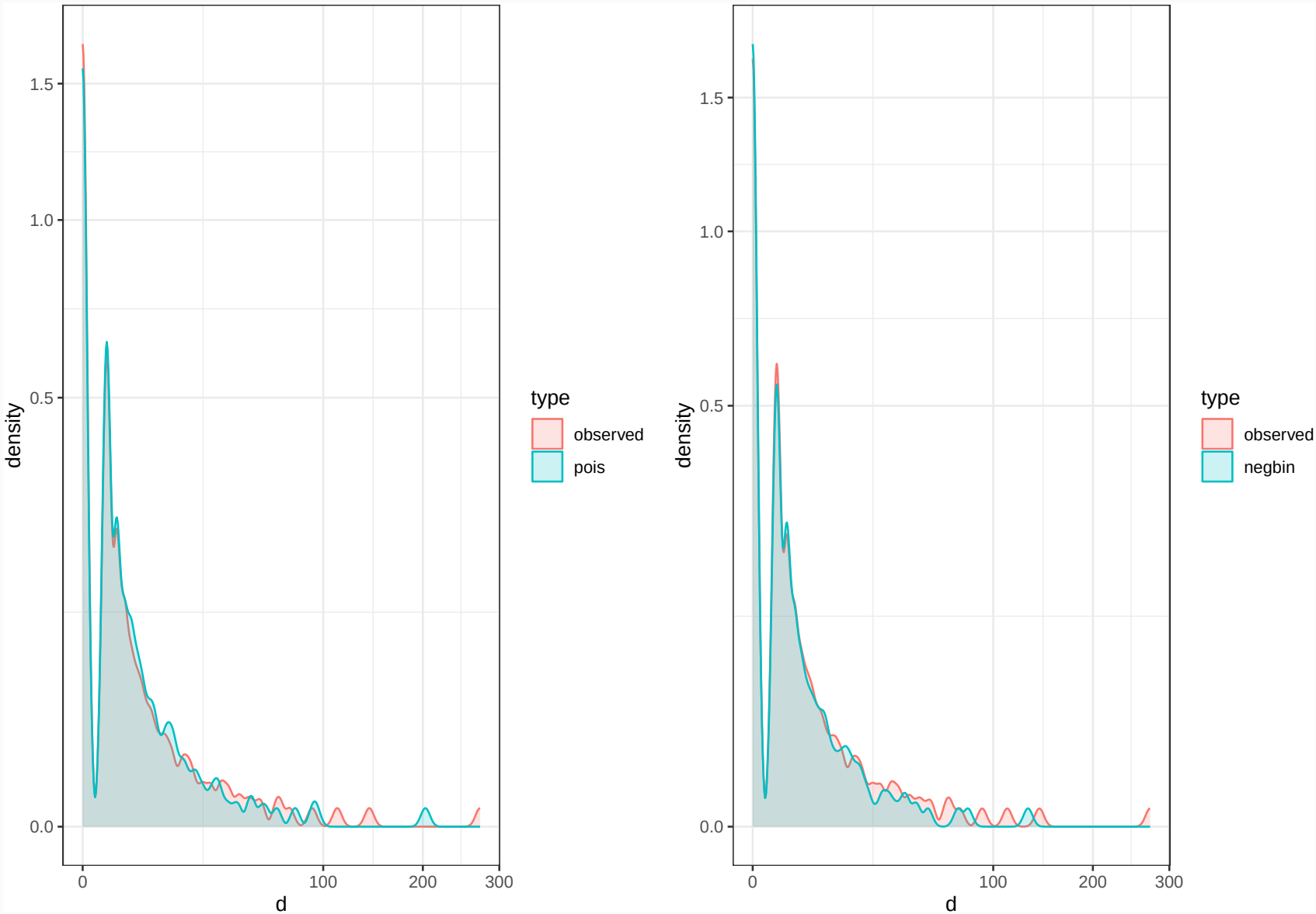




Examining the posteriors: Negative Binomial



How Negative Binomial influences inference



Posterior inference: Poisson

Recall that our Poisson model assumed

$$y \sim \text{Poisson}(\lambda_i)$$

$$\lambda_i = \beta_0 + \log p_i$$

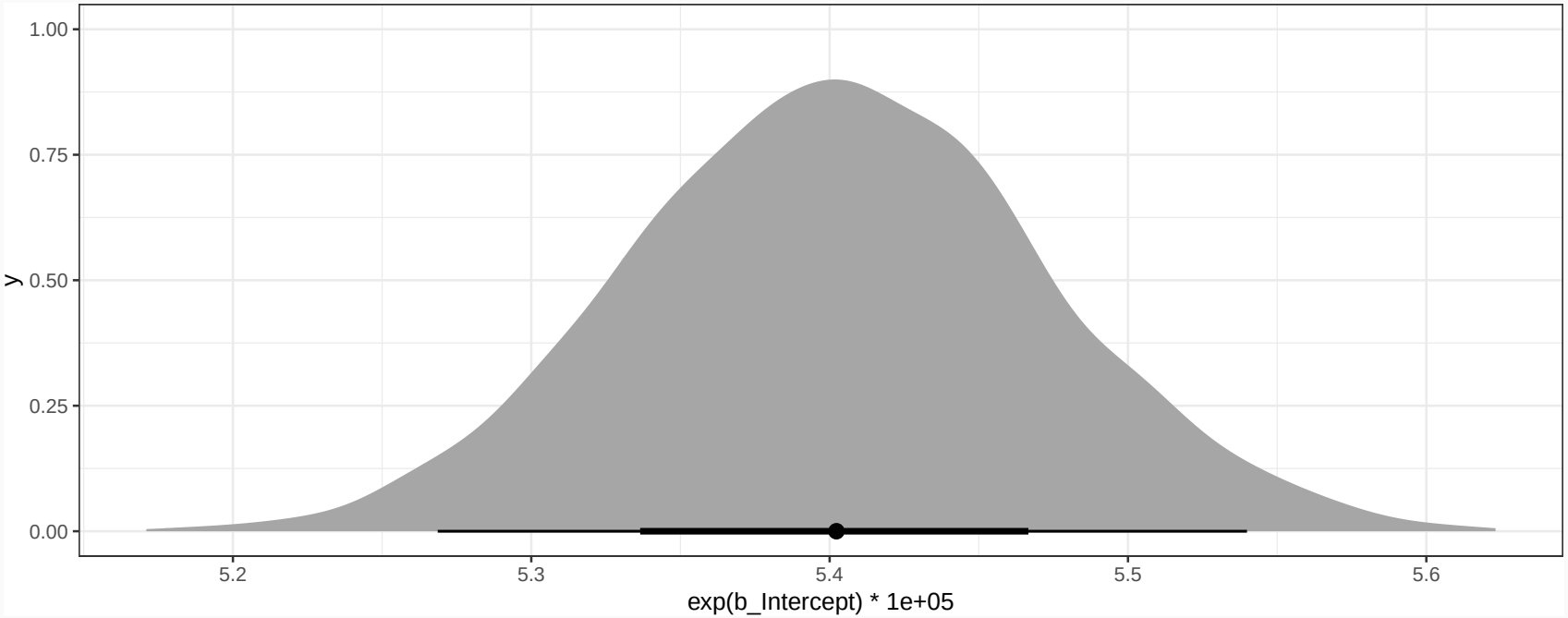
So what is λ?

```
posterior_interval(fe_0, variable = "b_Intercept")
```

```
                2.5%     97.5%
b_Intercept -9.851153 -9.800951
```

The posterior for lambda

```
post_pois <- as_draws_df(fe_0)
ggplot(post_pois, aes(x = exp(b_Intercept) * 1e5)) +
  stat_slabinterval()
```



Posterior inference: Negative Binomial

Our Negative Binomial model says that each county now gets its own λ.

But `brms` gives us this thing called `shape` ϕ

```
posterior_interval(fe_NB0, variable = c("b_Intercept", "shape"))
```

```
                2.5%     97.5%
b_Intercept -9.930995 -9.841385
shape        2.241602  3.090054
```

Posterior inference: Negative Binomial

Our Negative Binomial model says that each county now gets its own λ.

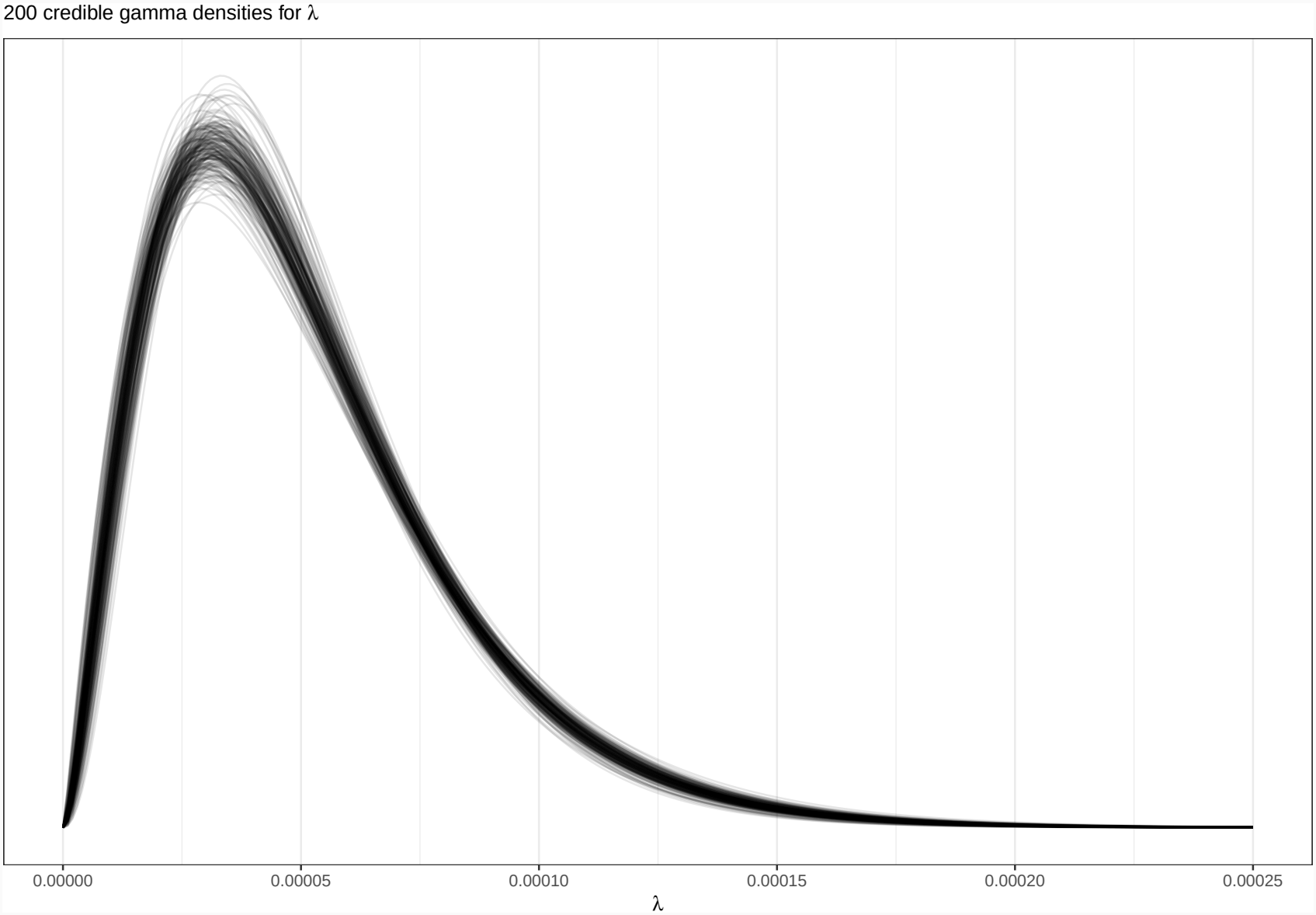
But `brms` gives us this thing called `shape` ϕ

```
posterior_interval(fe_NB0, variable = c("b_Intercept", "shape"))
```

```
                2.5%     97.5%
b_Intercept -9.930995 -9.841385
shape        2.241602  3.090054
```

There is an implied set of posterior Gamma distributions here, but we need to manipulate things a bit.

Many lambdas



Zero inflation

When there are more zeroes than your model expects

- Sometimes multiple processes are at play in count models: 1) determines whether the event occurs at all, and if so 2) determines the count of events
- If we are counting salamander populations in a forest, ordinary ecological variables like forest cover may predict population sizes. But another process, like pollutant exposure may determine whether there are any salamanders there at all.

The zero inflated model

This model contains two components.

1. A model describing the probability that the outcome is zero
2. A model describing the expected event count

Remember, regular count models also produce zeroes!

Possible zero inflated count outcomes

- Number of snow days in a school district
- Prison admissions in a county
- Well-visits to a doctor per person

How many fish did you catch on the camping trip?

I'm guessing that people who went camping along were more likely to go fishing. People in big groups and groups with kids were less likely to go fishing at all.

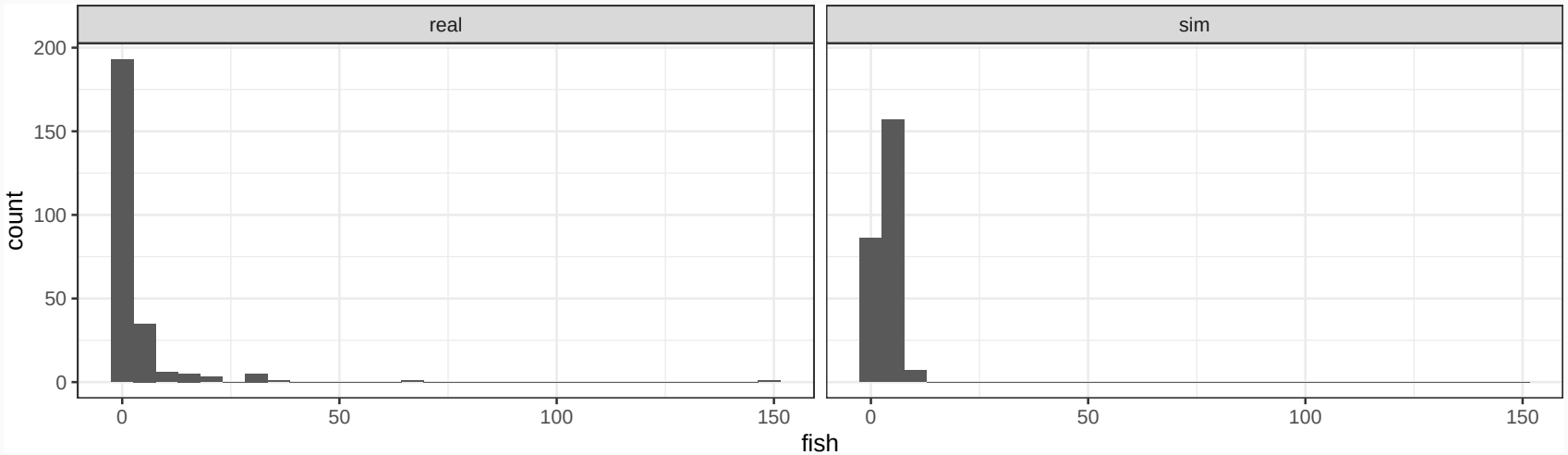
```
zinb <- read.csv("https://stats.idre.ucla.edu/stat/data/fish.csv") %>%
  select(persons, camper, child, count)
head(zinb)
```

```
  persons camper child count
1       1      0     0     0
2       1      1     0     0
3       1      0     0     0
4       2      1     1     0
5       1      0     0     1
6       4      1     2     0
```

Let's think about this

The mean number of fish caught in the data is 3.3, let's simulate a Poisson(3.3)

```
sim_fish<-rpois(250,3.3)
plot_dat<-data.frame(fish = c(sim_fish, zinb$count),
                     kind = rep(c("sim", "real"), each = 250))
ggplot(plot_dat,
       aes(x = fish)) +
  geom_histogram() +
  facet_wrap(~kind)
```



Let’s build our zero-inflated model

We suggest that kids and other people get in the way of fishing. If you go fishing, more people (rods) likely means more fish.

Let’s build our zero-inflated model

We suggest that kids and other people get in the way of fishing. If you go fishing, more people (rods) likely means more fish.

- First, we will run a logistic regression to estimate the probability that each group went fishing at all.
- We add this probability of not going fishing to our Poisson estimated probability of catching nothing when people did go fishing.
- Because these are separate linear models, we’ll get different parameters

Two likelihoods one model

$$fish_i \sim \text{Zero-inflated Negative Binomial}(p_i, \mu, \phi)$$

$$\text{logit}(p_i) = \alpha_p + \beta_{1p} \times kids_i + \beta_{2p} \times persons_i$$

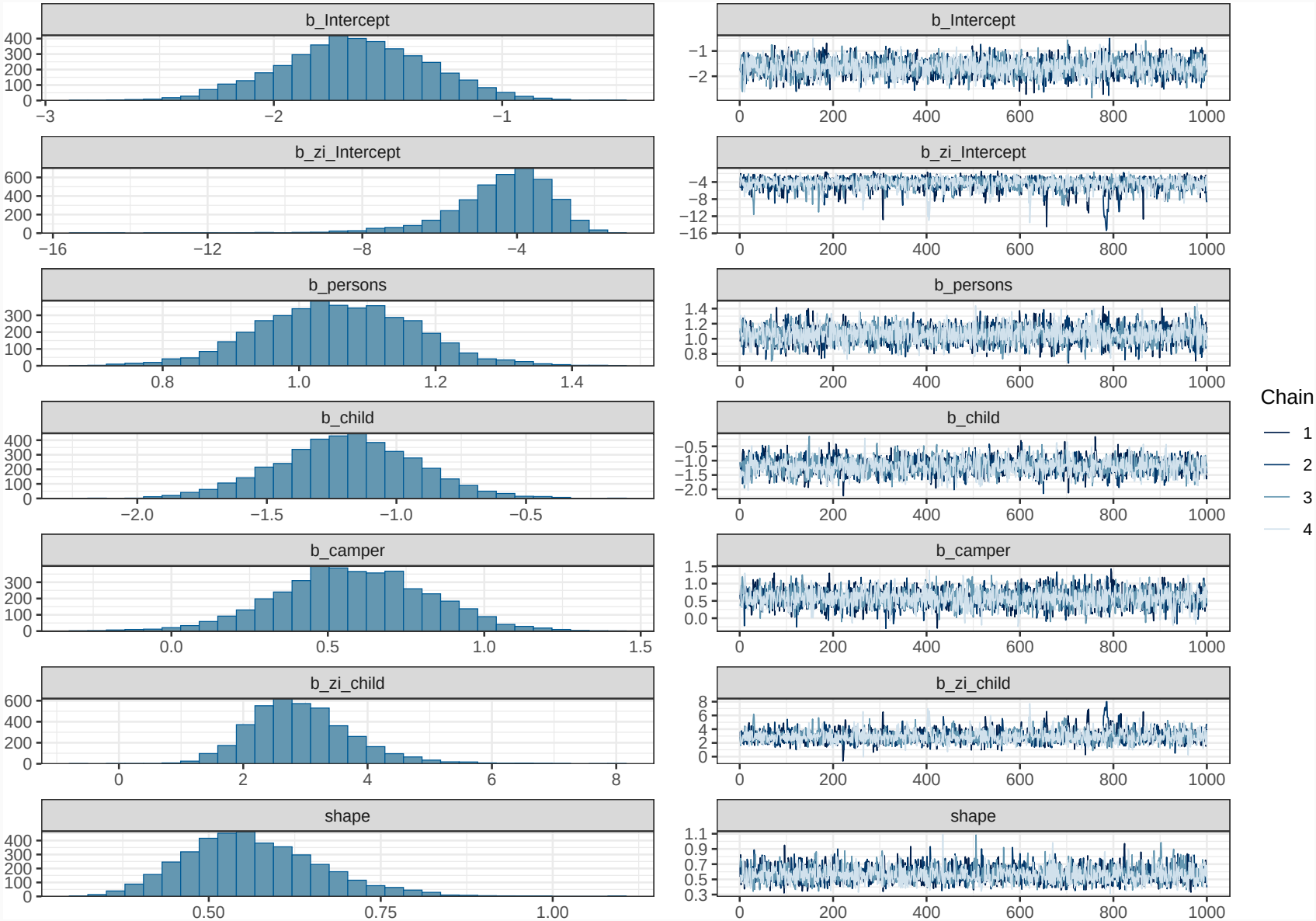
$$\log(\mu_i) = \alpha_\mu + \beta_{1\mu} \times persons_i$$

Estimation

We will use weakly informative priors on all parameters

```
priors <- c(set_prior("normal(0,5)", class= "b"),
            set_prior("normal(0,5)", class = "Intercept"),
            set_prior("exponential(1)", class = "shape"))
# bf is for complex brms formulas
mfish <- brm(bf(count~ persons + child + camper,
                zi~child),
             data = zinb,
             prior = priors,
               family = "zero_inflated_negbinomial")
```

Check convergence



Model output

```
mfish
```

```
 Family: zero_inflated_negbinomial
  Links: mu = log; zi = logit
Formula: count ~ persons + child + camper
         zi ~ child
   Data: zinb (Number of observations: 250)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
             Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept       -1.65      0.33    -2.28    -1.02 1.00     4178     2719
zi_Intercept    -4.42      1.42    -7.83    -2.42 1.00     1802     1381
persons          1.06      0.11     0.83     1.28 1.00     3861     2571
child           -1.19      0.27    -1.73    -0.66 1.00     2377     2489
camper           0.58      0.24     0.10     1.04 1.00     3086     2285
zi_child         2.95      0.86     1.52     4.87 1.00     2324     1497

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
shape     0.57      0.10     0.39     0.79 1.00     3154     3144

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

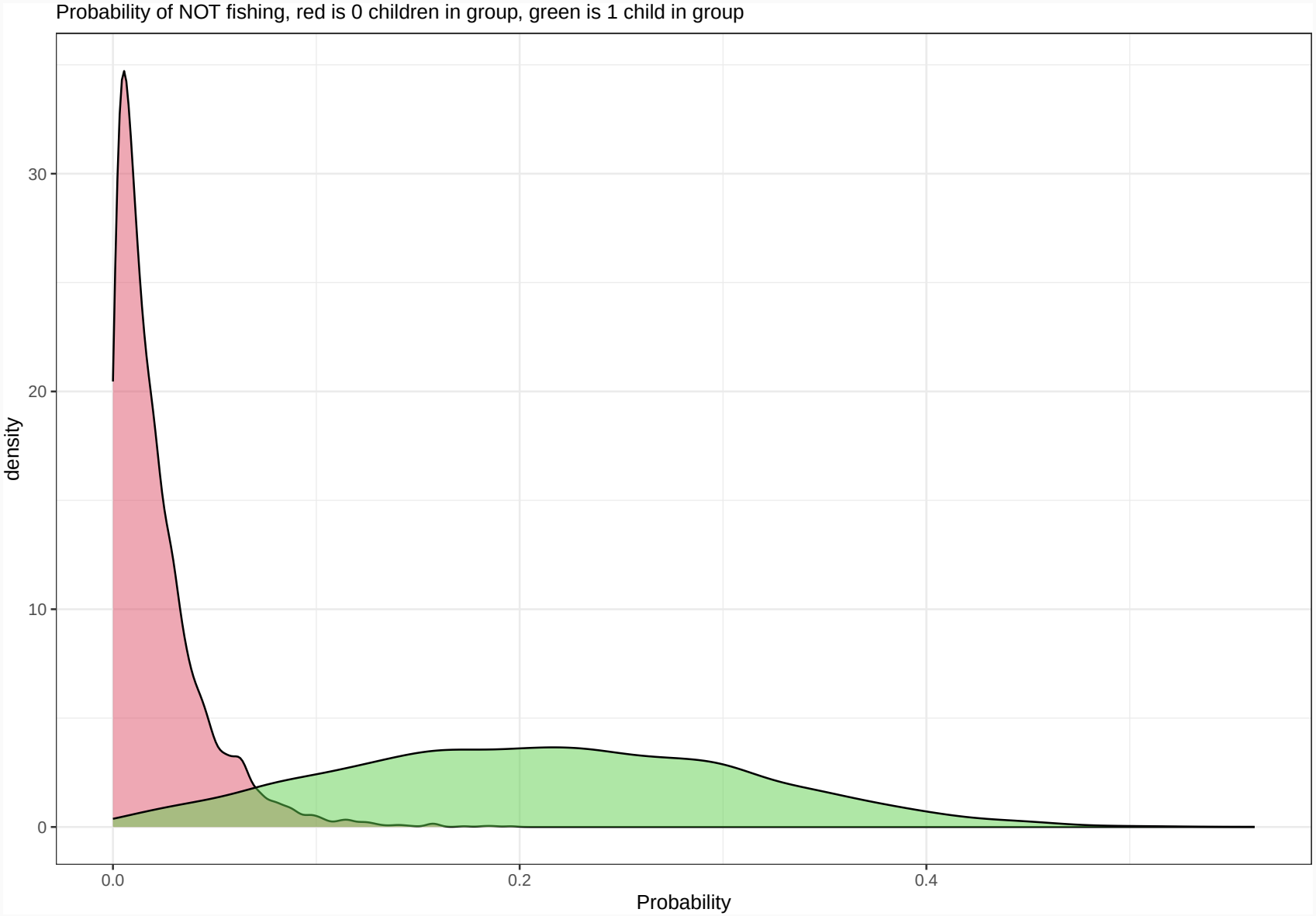
So what is the probability of NOT going fishing

Use the logit model for zeroes, campers with 1 child and no children

$$E(\text{Didn't fish}) = \text{Logit}^{-1}(\beta_{z0} + \beta_{z1} children)$$

```
mfish_post <- as_draws_df(mfish)
# inverse logit is 1/1(1+exp(-x)), or use logistic()
mfish_post <- mfish_post |>
  mutate(e0Logit_nokids = logistic(b_zi_Intercept),
         e0Logit_1kid =  logistic(b_zi_Intercept + b_zi_child))
```

Visualize


Probability of NOT fishing, red is 0 children in group, green is 1 child in group
density
30
20
10
0
0.0
0.2
0.4
Probability

Zeroes now have two sources

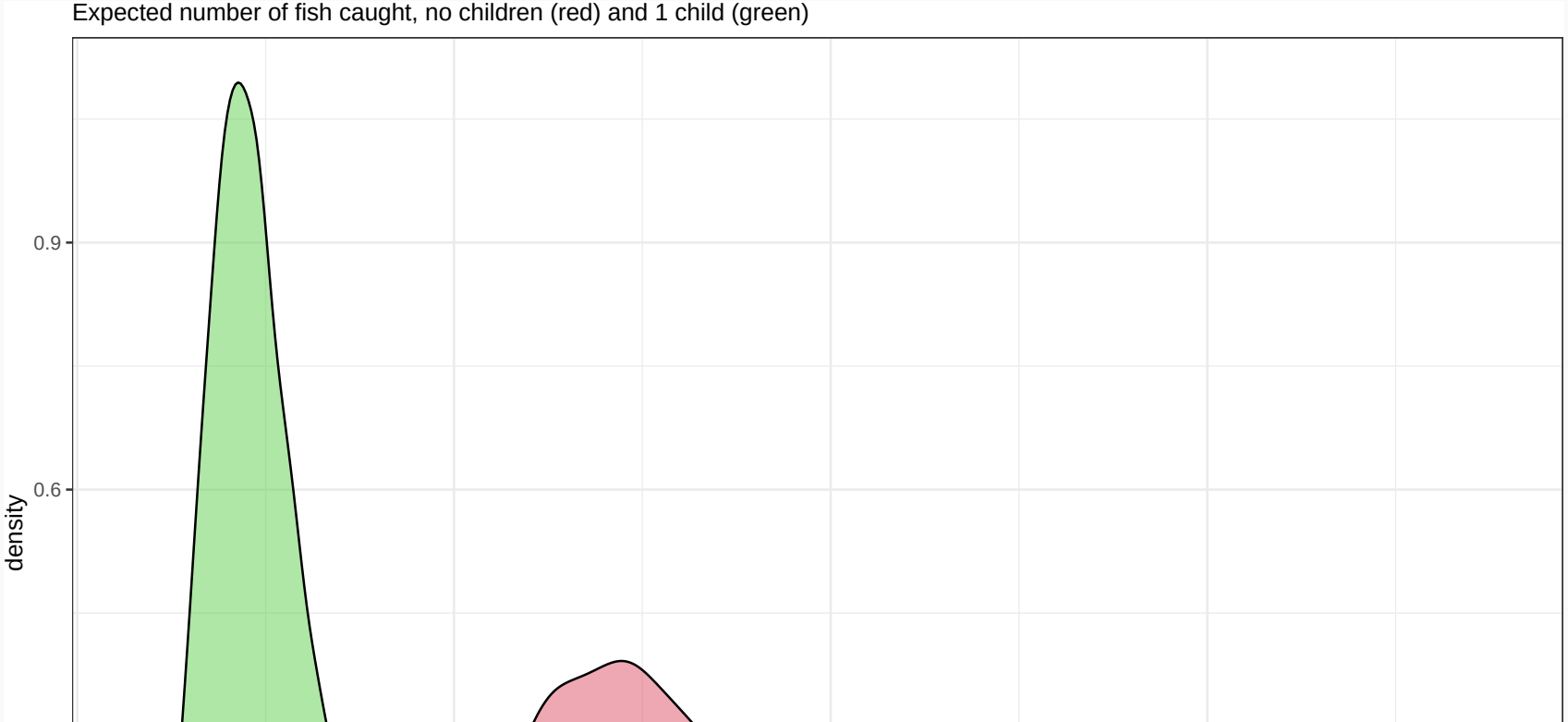
The Logit component models the probability of fishing, and the Negative Binomial (log link) models the count of fish (which can be zero!)

Let's pull apart the count component for the 1 and zero child groups again, now assuming 3 people in the group, and not in an RV (camper).

```
mfish_post <- mfish_post |>
  mutate(eNegBin_nokids = exp(b_Intercept + b_persons * 3),
         eNegBin_1kid =  exp(b_Intercept + b_child + b_persons * 3))
```

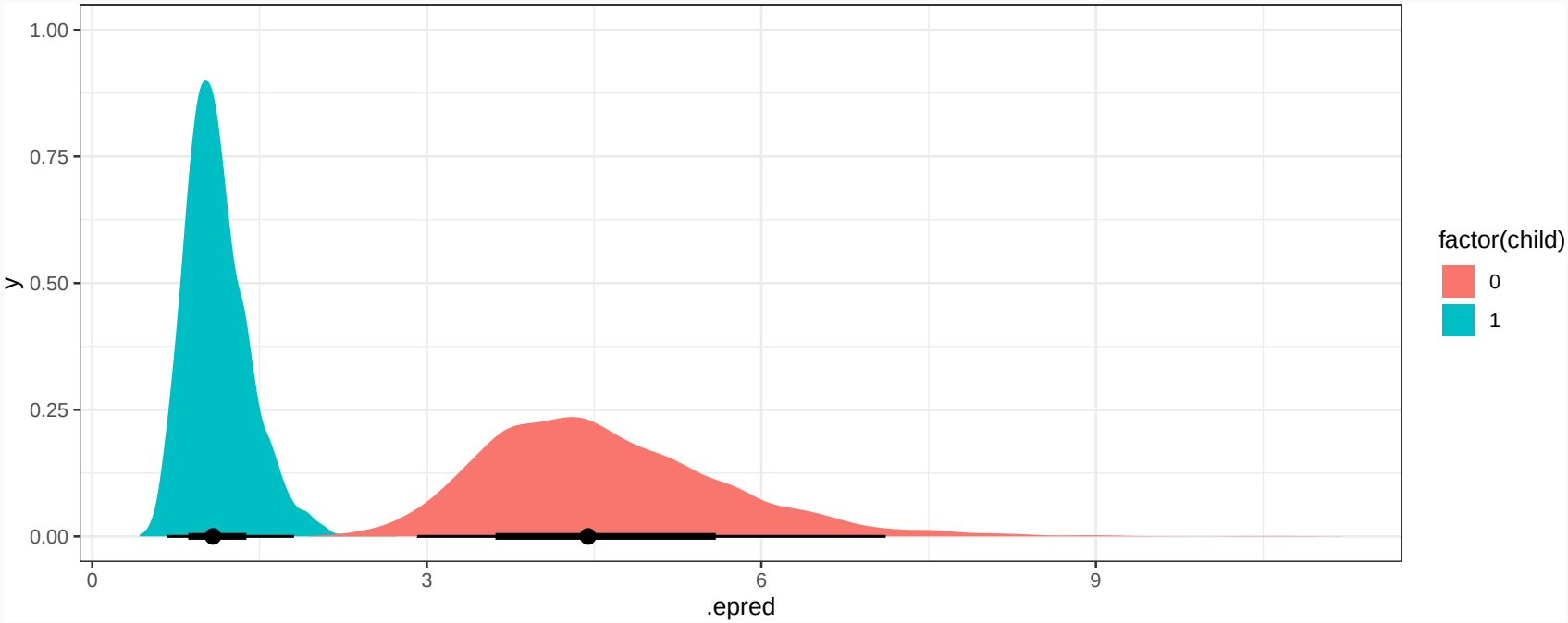
Visualize the posteriors

```
ggplot(mfish_post,
       aes(x = eNegBin_nokids)) +
  geom_density(fill = 2, alpha = 0.5) +
  geom_density(aes(x = eNegBin_1kid), fill = 3, alpha = 0.5) +
  labs(subtitle = "Expected number of fish caught, no children (red
       x = "Fish caught")
```



Full model visualization: expected fish

```
plotdat <- expand_grid(persons = 3, camper = 0, child = c(0,1)) |>
  add_epred_draws(mfish)
ggplot(plotdat, aes(x = .epred, fill = factor(child))) +
  stat_slabinterval()
```



Full model visualization: predicted fish

```
plotdat <- expand_grid(persons = 3, camper = 0, child = c(0,1)) |>
  add_predicted_draws(mfish)
ggplot(plotdat, aes(x = .prediction, fill = factor(child))) +
  stat_histinterval(alpha = 0.5, .width = 0) +
  scale_y_sqrt() +
  scale_x_sqrt()
```

